



SRI KRISHNA COLLEGE OF ENGINEERING AND TECHNOLOGY

An Autonomous Institution | Approved by AICTE | Affiliated to Anna University | Accredited by NAAC with A++ Grade
Kuniamuthur, Coimbatore – 641008
Phone : (0422)-2678001 (7 Lines) | Email : info@skcet.ac.in | Website : www.skcet.ac.in

PLANT CARE SCHEDULER

A PROJECT REPORT

Submitted by

SUCHITRA K M

In the partial fulfilment for the award of the degree

of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE AND DESIGN

SRI KRISHNA COLLEGE OF ENGINEERING AND TECHNOLOGY

An Autonomous Institution | Approved by AICTE | Affiliated to Anna University |

Accredited by NAAC with A++ Grade

Kuniyamuthur, Coimbatore – 641008.

2024-2028



SRI KRISHNA COLLEGE OF ENGINEERING AND TECHNOLOGY

An Autonomous Institution | Approved by AICTE | Affiliated to Anna University | Accredited by NAAC with A++ Grade
Kuniamuthur, Coimbatore – 641008
Phone : (0422)-2678001 (7 Lines) | Email : info@skcet.ac.in | Website : www.skcet.ac.in

BONAFIDE CERTIFICATE

Certified that this project report “**PLANT CARE SCHEDULER**” is the bonafide work of “**SUCHITRA K M**” who carried out the project work under my supervision.

SIGNATURE

DR. F. MARGRET SHARMILA

HEAD OF THE DEPARTMENT

Department of Computer Science

and Design

Sri Krishna College of Engineering

and Technology

SIGNATURE

<NAME>

SUPERVISOR

Department of Computer Design

and Design

Sri Krishna College of Engineering

and Technology

Submitted for the Project viva-voce examination held on _____.

INTERNAL EXAMINER

EXTERNAL EXAMINER

ABSTRACT

Plant Care Scheduler is a full-stack web application developed to assist plant owners in organizing and managing their daily plant care activities through a centralized digital platform. The application enables users to maintain detailed plant records, generate personalized care schedules, monitor environmental conditions, track plant health, interact with a gardening community, and request consultations from plant care specialists.

The application is developed using React for the frontend and Java 24 with **Spring Boot** for the backend, following a layered architecture consisting of Controller, Service, Repository, and Database layers. Spring Data JPA and Hibernate are utilized for Object Relational Mapping (ORM), while MySQL is used as the relational database management system for persistent data storage.

The backend follows a **DTO–Mapper** approach to facilitate efficient data transfer between the client and server while maintaining a clear separation between the API and persistence layers.

The application comprises ten major modules, namely User Management, Plant Management, Species Management, Care Task Management, Environmental Data Management, Health Record Management, Community Post Management, Comment Management, Notification Management, and Consultation Management. These modules are exposed through **RESTful APIs** supporting Create, Read, Update, and Delete (CRUD) operations and are validated using Postman before integration with the frontend.

By combining scheduling, environmental monitoring, health tracking, community interaction, and specialist consultation, the application offers an efficient platform for managing plant care while providing a strong foundation for future enhancements.

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	iii
	TABLE OF CONTENTS	iv
	LIST OF FIGURES	vi
	LIST OF SYMBOLS, ABBREVIATIONS, AND NOMENCLATURE	vii
1	INTRODUCTION	1
1.1	Introduction	1
1.2	Problem Statement	1
1.3	Objectives	1
1.4	Scope of the Project	2
2	SYSTEM ANALYSIS AND REQUIREMENTS	3
2.1	Existing System	3
2.2	Limitations of the Existing System	3
2.3	Proposed System	4
2.4	Functional Requirements	6
2.5	Non-Functional Requirements	7
2.6	Summary of Requirements	8
3	SYSTEM DESIGN	9

3.1	System Architecture	9
3.2	UML Class Diagram	11
3.3	Entity Relationship Diagram	13
3.4	Activity Diagram	15
3.5	Sequence Diagram	17
4	IMPLEMENTATION, TESTING AND RESULTS	19
4.1	Technology Stack	19
4.2	System Implementation	19
4.3	Security Implementation	21
4.4	Testing and Results	21
5	CONCLUSION AND FUTURE ENHANCEMENTS	26
5.1	Conclusion	26
5.2	Future Enhancements	26

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE NO.
Figure 3.1	UML Class Diagram of Plant Care Scheduler	11
Figure 3.2	Entity Relationship Diagram of Plant Care Scheduler	13
Figure 3.3	Plant Management Activity Diagram	15
Figure 3.4	Care Task Activity Diagram	16
Figure 3.5	Community Post Activity Diagram	16
Figure 3.6	Care Task Sequence Diagram	17
Figure 3.7	Community Post Sequence Diagram	18
Figure 4.1	Folder Structure	23
Figure 4.2	API Documentation	24
Figure 4.3	Portal testcases for Plant Care Scheduler Application	25

LIST OF SYMBOLS, ABBREVIATIONS, AND NOMENCLATURE

ABBREVIATION	EXPANSION
API	Application Programming Interface
CRUD	Create, Read, Update, Delete
ER	Entity Relationship
HTTP	Hypertext Transfer Protocol
JPA	Java Persistence API
JSON	JavaScript Object Notation
JWT	JSON Web Token
LMS	Learning Management System
ORM	Object Relational Mapping
REST	Representational State Transfer
SRS	Software Requirements Specification
SQL	Structured Query Language
UI	User Interface
UML	Unified Modeling Language
URL	Uniform Resource Locator

CHAPTER 1: INTRODUCTION

1.1 Introduction

Plant care requires regular attention to watering, sunlight, fertilization, and health monitoring. Managing these tasks manually can be difficult, especially for users with multiple plants. Plant Care Scheduler is a web application that helps users manage plant profiles, care schedules, environmental data, plant health, community discussions, and specialist consultations in one place.

The system uses React for the frontend and Spring Boot for the backend. It follows a layered architecture with Controller, Service, Repository, and Database layers. Spring Data JPA, Hibernate, and MySQL are used for data management, and the backend provides RESTful APIs for integration with other applications.

1.2 Problem Statement

Plant care is often managed using notebooks, calendar reminders, spreadsheets, or simple mobile applications that focus only on basic scheduling. These approaches fail to provide an integrated solution for managing plant information, care routines, environmental conditions, health assessments, and expert guidance. As the number of plants increases, owners find it difficult to maintain consistent care schedules, monitor plant health, identify problems at an early stage, and keep historical records. Additionally, there is limited support for sharing knowledge with other gardeners or seeking professional assistance when plants develop diseases or growth issues.

Plant Care Scheduler addresses these challenges by providing a centralized and secure platform that combines plant management, automated care task scheduling, environmental data recording, health monitoring, community interaction, specialist consultation, and notification services within a single system. By consolidating all plant-related information and activities, the platform helps users maintain healthier plants, make informed care decisions, and improve the overall efficiency of plant care management.

1.3 Objectives

- To develop a centralized application for managing plant care activities efficiently.
- To maintain detailed records of plants, species information, environmental conditions, and health history.
- To automate the scheduling and tracking of plant care tasks.
- To provide RESTful APIs using Spring Boot for seamless frontend integration.

- To facilitate community interaction through posts and comments related to plant care.
- To enable users to request and manage consultations with plant care specialists.
- To provide a scalable backend architecture using Spring Boot, Spring Data JPA, Hibernate, and MySQL.

1.4 Scope of The Project

Plant Care Scheduler is a full-stack application designed to simplify plant care management through a structured and intuitive platform. It allows users to organize plant details, plan care activities, monitor environmental conditions, maintain health records, engage in community discussions, and seek guidance from specialists when needed.

The system provides RESTful APIs for the major functional modules and is integrated with a React-based frontend. Built on a modular architecture, it is prepared for future improvements such as image uploads, automated reminders, advanced search and filtering, and scheduled background tasks, making it a strong foundation for a scalable plant care management solution.

CHAPTER 2: SYSTEM ANALYSIS AND REQUIREMENTS

2.1 Existing System

Plant care is often managed through manual reminders, notebooks, spreadsheets, or simple mobile applications. These methods help users track routine activities.

However, they usually do not provide an integrated way to manage all plant care needs in one place. Features such as environmental monitoring, health record maintenance, community interaction, and specialist consultation are usually missing. As the number of plants increases, manual tracking becomes more difficult. Users may forget watering schedules, fertilization dates, sunlight needs, or health observations. This can result in missed care activities, inconsistent maintenance, and incomplete plant records.

2.2 Limitations of The Existing System

The major limitations of traditional plant care methods and existing plant management applications are:

- **Manual Plant Care Management:** Plant owners often rely on memory, handwritten notes, or simple reminders for watering, fertilizing, pruning, and other maintenance tasks, leading to missed or inconsistent care.
- **Fragmented Plant Information:** Plant profiles, care schedules, health records, environmental conditions, and growth history are typically maintained across multiple applications or not recorded at all, making plant management inefficient.
- **Limited Health Monitoring:** Most existing solutions provide only basic reminders and do not maintain structured health records, making it difficult to identify recurring diseases, pest infestations, or growth-related issues.
- **Lack of Expert Consultation:** Plant owners often have no direct platform to seek guidance from gardening specialists, delaying the diagnosis and treatment of plant health problems.
- **Poor Community Collaboration:** Existing systems rarely provide an integrated platform where plant enthusiasts can share experiences, ask questions, discuss plant care techniques, and receive feedback from other users.

- **Limited Security and User Management:** Many applications lack robust authentication and authorization mechanisms, exposing user information and restricting secure role-based access for administrators and specialists.
- **Difficulty in Task Scheduling:** Managing multiple plants with different watering frequencies, fertilizing intervals, sunlight requirements, and maintenance activities becomes increasingly difficult without automated scheduling.
- **Limited Scalability:** Traditional or manually managed systems become inefficient as the number of users, plants, care activities, consultations, and community interactions grows.

2.3 Proposed System

The proposed **Plant Care Scheduler** is a centralized and secure backend application designed to simplify plant management through intelligent scheduling, health tracking, community collaboration, and expert consultation. The application provides RESTful APIs for managing users, plant profiles, plant species, care tasks, environmental data, health records, consultations, notifications, community posts, and comments.

The system supports multiple user roles, including **USER, SPECIALIST, and ADMIN**, with role-based authorization ensuring secure access to application resources. Plant owners can manage their plants, monitor care schedules, maintain health records, request consultations, and participate in community discussions. Specialists can review consultation requests, provide guidance, and maintain plant health recommendations, while administrators oversee overall system management. The application incorporates **Spring Security with JWT authentication** to secure all protected REST APIs. During login, authenticated users receive a JWT access token that must be included in the **Authorization** header for subsequent requests. User passwords are securely stored using **BCrypt hashing**, ensuring that sensitive credentials are never saved in plain text.

The system also provides structured management of plant care activities by maintaining personalized schedules, recording environmental conditions, tracking plant health history, sending notifications for upcoming tasks, and enabling community knowledge sharing through discussion forums. This centralized architecture improves data consistency, enhances security, simplifies plant maintenance, and provides a scalable foundation for future intelligent features such

as AI-based plant disease detection, smart care recommendations, and IoT sensor integration.

Comparison Of Existing and Proposed System

Feature	Existing System	Proposed System
Plant Care Management	Manual tracking using notebooks, reminders, or multiple apps	Centralized plant care management platform
Plant Health Monitoring	No structured method to maintain plant health history	Dedicated health records with diagnosis and treatment tracking
Care Scheduling	Manual reminders for watering, fertilizing, pruning, and repotting	Automated care task scheduling and tracking
Environmental Monitoring	Environmental conditions are rarely recorded systematically	Stores and monitors plant environmental data for better care decisions
Expert Consultation	Users depend on external sources or local nurseries	Built-in consultation management between users and plant specialists
Community Support	Knowledge shared through scattered social media or forums	Integrated community posts and comments within the application
Authentication & Security	Basic or no authentication	JWT-based authentication with Spring Security and BCrypt password encryption
Data Management	Information stored across notebooks, spreadsheets, or multiple applications	Centralized relational database using Spring Boot and MySQL
Scalability	Difficult to manage as the number of users and plants increases	Layered architecture that supports future expansion and additional features

2.4 Functional Requirements

Functional requirements define the operations and services that the Plant Care Scheduler System must provide. The major functional requirements are described below.

2.4.1 User Management

The system shall allow users to register and authenticate securely. Each user is assigned a role such as USER or SPECIALIST. User information, including username, email, role, account status, gardening experience, notification preferences, and last login, is maintained by the system. Passwords are securely stored using BCrypt hashing.

2.4.2 Plant Management

The system shall allow users to add, update, retrieve, and delete plant records. Each plant record stores details such as plant name, species, planting date, watering frequency, sunlight requirements, fertilizer schedule, and ownership information. Every plant is associated with its respective owner.

2.4.3 Species Management

The system shall maintain a centralized database of plant species. Species information includes common name, scientific name, care difficulty, light requirements, watering frequency, humidity range, temperature range, soil pH, fertilizer schedule, pruning frequency, repotting interval, common diseases, and care tips. Users can associate their plants with predefined species information.

2.4.4 Care Task Management

The system shall generate and manage plant care tasks such as watering, fertilizing, pruning, repotting, and other maintenance activities. Each task stores information including task type, scheduled date, completion status, completion time, and the associated plant. Users can update task status after completing plant care activities.

2.4.5 Health Record Management

The system shall allow users to record plant health observations, including symptoms, diagnosis, treatment recommendations, specialist consultation details, and observation dates. Specialists can update health records by providing expert recommendations for plant diseases and pest management.

2.4.6 Environmental Data Management

The system shall maintain environmental information such as temperature, humidity, soil moisture, and light intensity for individual plants. These records help users monitor growing conditions and make informed plant care decisions.

2.4.7 Community and Comment Management

The system shall provide a community platform where users can create posts, share plant care experiences, ask questions, and exchange gardening knowledge. Other users can comment on posts, enabling collaborative discussions and community engagement.

2.4.8 Consultation Management

The system shall allow users to request consultations with plant care specialists. Consultation records include appointment date, problem description, consultation status, meeting link, and specialist assignment. Specialists can manage consultation requests and provide guidance.

2.4.9 Notification Management

The system shall generate notifications related to scheduled care tasks, consultation updates, community interactions, and important plant care reminders. Users can view notification history and manage notification preferences.

2.5 Non-Functional Requirements

2.5.1 Security

The system shall use Spring Security for application security. JWT-based authentication shall protect secured REST APIs, while BCrypt shall be used for password hashing. Role-based authorization shall restrict access according to the authenticated user's role (e.g., USER or SPECIALIST).

2.5.2 Performance

The system should process REST API requests efficiently and provide acceptable response times for common operations such as managing plants, updating care tasks, recording health observations, and participating in community discussions.

2.5.3 Scalability

The layered architecture should support increasing numbers of users, plants, consultations, community posts, and health records without requiring significant architectural changes.

2.5.4 Maintainability

The system shall follow a structured architecture consisting of Controller, Service, Repository, Security, and Database layers. This separation of responsibilities improves readability, testing, maintenance, and future development.

2.5.5 Reliability and Data Persistence

Application data shall be stored persistently in a MySQL database using Spring Data JPA and Hibernate. The system should maintain consistent records for users, plants,

species, care tasks, consultations, notifications, community posts, comments, environmental data, and health records.

2.5.6 Usability and API Documentation

The REST APIs shall be documented using Swagger/OpenAPI, allowing developers and testers to understand and interact with available endpoints through a user-friendly interface.

2.6 Summary of Requirements

Requirement Category	Key Requirement
Authentication	JWT-based secure login
Authorization	USER and SPECIALIST roles
Password Security	BCrypt password hashing
User Management	User registration and profile management
Plant Management	Add, update, retrieve, and delete plant records
Species Management	Centralized species information
Care Task Management	Schedule and manage plant care tasks
Health Management	Maintain plant health records
Environmental Monitoring	Record plant environmental conditions
Community	Posts and comments
Consultation	Specialist consultation management
Notifications	Task and consultation reminders
Data Storage	MySQL with JPA/Hibernate
API Documentation	Swagger/OpenAPI
Architecture	Controller → Service → Repository → Database

CHAPTER 3: SYSTEM DESIGN

3.1 System Architecture

The Plant Care Scheduler System follows a layered architecture that separates the application into multiple functional layers. This architectural pattern improves modularity, maintainability, scalability, readability, and security by assigning a specific responsibility to each layer. Every layer communicates with the adjacent layer in a structured manner, ensuring clean separation of concerns and efficient request processing.

Architectural Layers

The overall architecture of the application consists of the Client/Swagger Interface, Security Layer, Controller Layer, Service Layer, Repository Layer, and MySQL Database.

3.1.1 Client and Swagger Interface

The client represents any application, such as a web frontend or third-party system, that communicates with the backend using HTTP requests. During development and testing, Swagger UI (Open API) provides an interactive interface for accessing and testing RESTful API endpoints. Data is exchanged between the client and server primarily in JSON format.

3.1.2 Security Layer

The Security Layer safeguards the application against unauthorized access. **Spring Security** is used to manage authentication and authorization, while **JWT (JSON Web Token)** provides stateless authentication for securing protected REST APIs.

The **JwtAuthenticationFilter** intercepts incoming requests, extracts the Bearer token from the Authorization header, validates the token, loads the authenticated user details, and establishes the security context before allowing access to protected resources.

Role-Based Access Control (RBAC) is implemented to provide different levels of system access for:

- **USER**
- **SPECIALIST**

User passwords are securely stored using **BCrypt password hashing**, ensuring sensitive credentials are protected.

3.1.3 Controller Layer

The Controller Layer receives incoming HTTP requests and maps them to the appropriate REST API operations. Controllers validate request data, invoke the corresponding service methods, and return standardized HTTP responses to the client.

3.1.4 Service Layer

The Service Layer contains the core business logic of the application. It acts as an intermediary between the Controller and Repository layers by processing requests, enforcing business rules, validating data, managing plant care schedules, handling consultations, maintaining health records, managing community interactions, and coordinating notification services.

All database-related operations are delegated to the Repository Layer.

The overall processing flow is:

Controller → Service → Repository

3.1.5 Repository Layer

The Repository Layer is responsible for all database interactions. Repository interfaces extend **JpaRepository**, allowing Spring Data JPA to automatically provide common CRUD operations without requiring manual SQL queries.

Frequently used operations include:

- **save()**
- **findAll()**
- **findById()**
- **deleteById()**
- **existsById()**

Custom query methods are also used for operations such as retrieving plants by owner, searching species, fetching care tasks, consultation records, notifications, community posts, and comments.

3.1.6 Database Layer

The Database Layer uses **MySQL** as the relational database management system to store and manage all plant care information. **Hibernate**, through Spring Data JPA, maps Java entities to database tables and handles object-relational mapping (ORM).

The database stores information related to:

- Users
- Plants

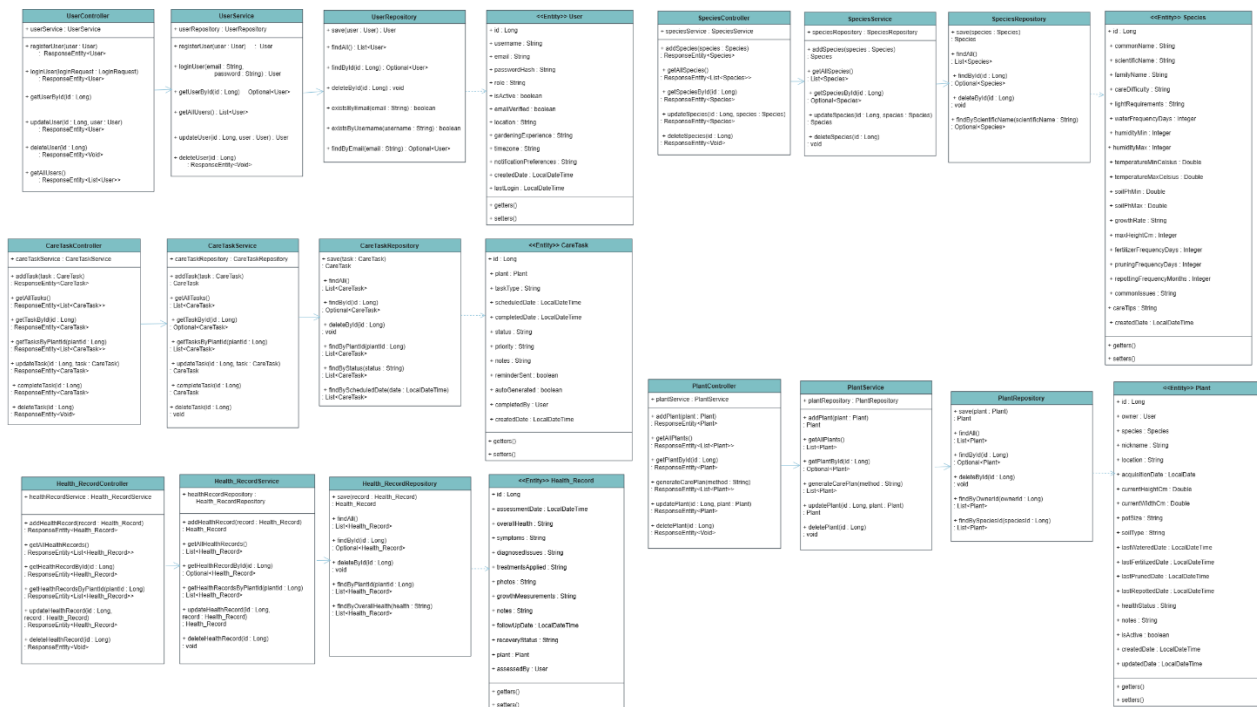
- Species
- Care Tasks
- Health Records
- Environmental Data
- Community Posts
- Comments
- Consultations
- Notifications

This layered architecture provides a secure, scalable, and maintainable foundation for managing the complete plant care lifecycle, including scheduling, monitoring, health tracking, expert consultation, and community collaboration.

3.2 UML Class Diagram

The UML Class Diagram represents the static structure of the Plant Care Scheduler application. It illustrates the major classes, their attributes, methods, and relationships between different modules. The diagram provides an overview of how the application's entities collaborate to manage plant care, health monitoring, environmental tracking, consultations, notifications, and community interactions.

Figure 3.1: UML Class Diagram

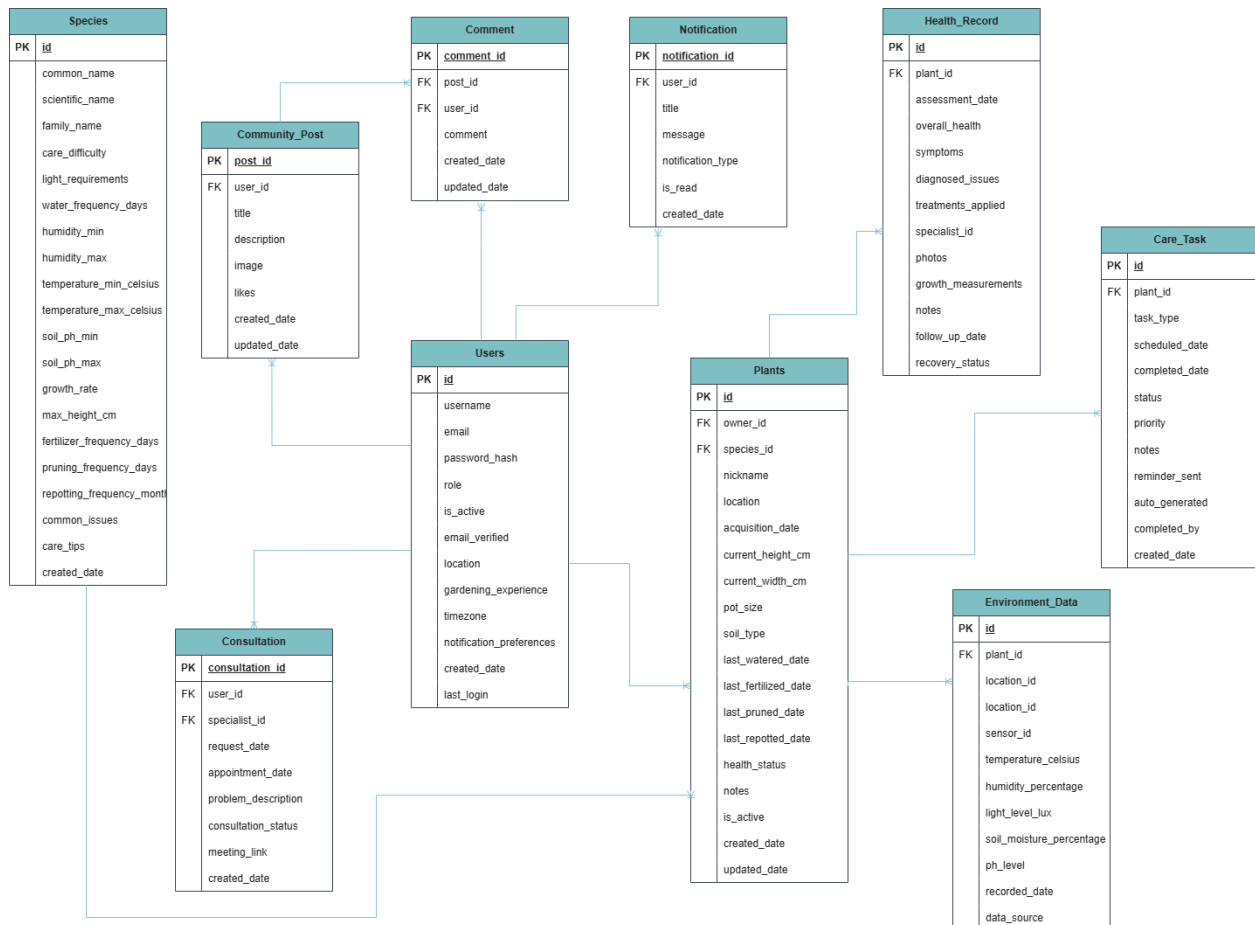


- **Consultation:** Manages consultation requests between users and plant specialists. It stores appointment details, meeting links, consultation status, and problem descriptions.
- **Notification:** Stores reminders and notifications generated for users regarding care tasks, consultations, and community activities.

3.3 Entity Relationship Diagram

The Entity Relationship (ER) Diagram represents the logical database structure of the PlantCareScheduler system. It illustrates the major database entities, their primary keys, foreign keys, attributes, and relationships. The database is designed to manage plant information, care scheduling, environmental monitoring, health assessments, consultations, community discussions, and notifications.

Figure 3.2: Entity Relationship Diagram



The **User** entity stores account information including username, email, encrypted password, role, location, gardening experience, notification preferences, account

status, and last login. A user can own multiple plants, create community posts, add comments, receive notifications, and request consultations.

The **Plant** entity stores information about each plant including nickname, species, owner, location, health status, watering history, fertilizing history, pruning history, soil type, and growth measurements. Every plant belongs to one user and one species.

The **Species** entity maintains predefined botanical information such as scientific name, watering frequency, sunlight requirements, humidity range, soil pH, fertilizer frequency, pruning schedule, growth rate, and care recommendations. A species can be associated with multiple plants.

The **CareTask** entity stores scheduled maintenance activities such as watering, fertilizing, pruning, and repotting. Each task belongs to one plant and stores scheduling information, reminders, priority, completion status, and notes.

The **HealthRecord** entity records periodic health assessments of plants. It stores symptoms, diagnosis, treatment details, recovery status, specialist observations, photographs, and follow-up schedules. Each health record belongs to one plant and one specialist.

The **EnvironmentData** entity stores environmental measurements including temperature, humidity, light intensity, soil moisture, pH level, sensor information, and recording timestamps. Multiple environmental records can be associated with one plant.

The **CommunityPost** entity stores plant-related posts created by users, while the **Comment** entity stores user comments associated with community posts.

The **Consultation** entity manages appointment requests between users and specialists, storing appointment dates, meeting links, consultation status, and problem descriptions.

The **Notification** entity stores reminders and alerts sent to users for scheduled tasks, consultations, and community activities.

Major Database Relationships

- One User can own multiple Plants.
- One Species can be associated with multiple Plants.
- One Plant can contain multiple Care Tasks.
- One Plant can contain multiple Health Records.
- One Plant can contain multiple Environment Data records.
- One User can create multiple Community Posts.
- One Community Post can contain multiple Comments.
- One User can create multiple Comments.
- One User can receive multiple Notifications.
- One User can request multiple Consultations.
- One Specialist can manage multiple Consultations.

3.4 Activity Diagram:

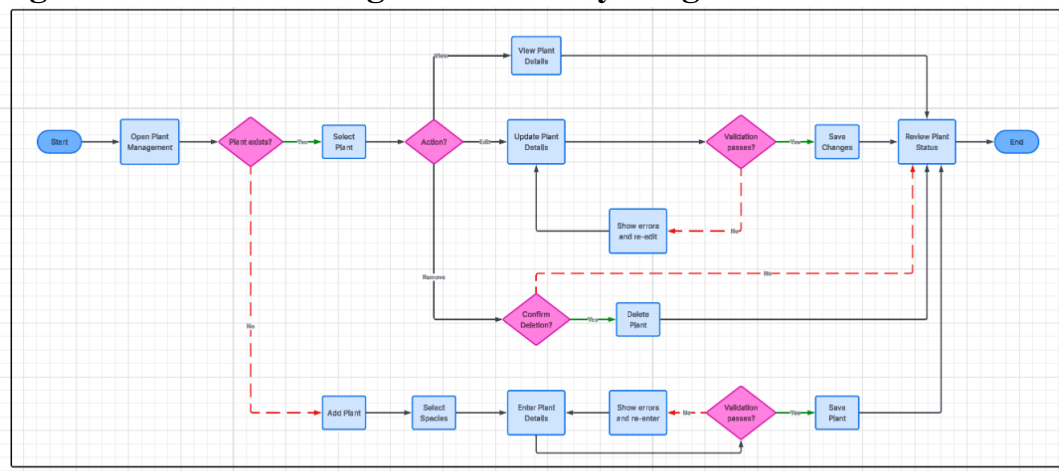
The workflow of the Plant Care Scheduler application is represented using three primary activity diagrams: **Plant Management**, **Care Task Management**, and **Community Post Management**. These diagrams describe how users interact with the system to perform major operations.

3.4.1 Plant Management Activity Diagram

This diagram illustrates how users manage their plants.

- **Workflow:** The user opens the Plant Management module and either selects an existing plant or adds a new one.
- **System Actions:** The system validates plant information before saving it. Users can also update existing plant details or delete unwanted plants after confirmation.

Figure 3.3: Plant Management Activity Diagram

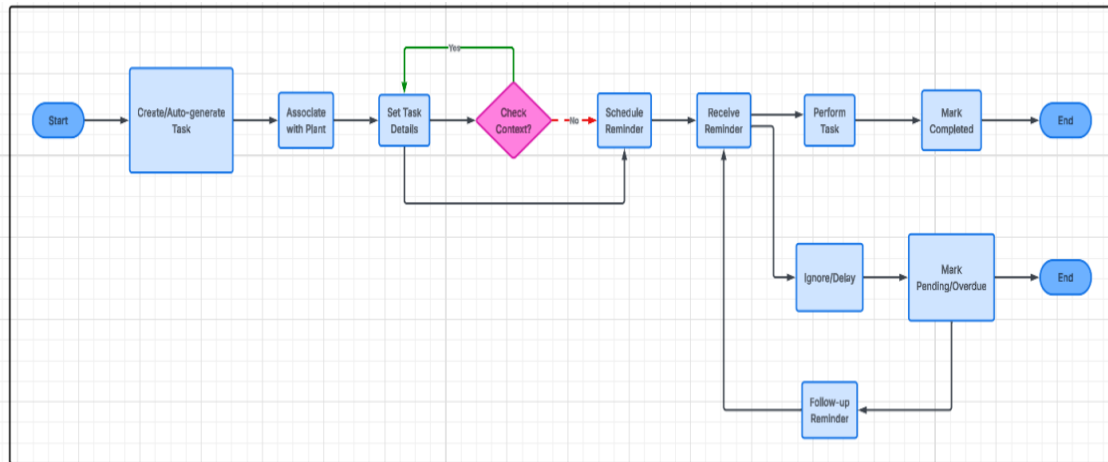


3.4.2 Care Task Activity Diagram

This diagram illustrates how plant care tasks are created and managed.

- **Workflow:** The user creates or auto-generates a care task and associates it with a plant.
- **Decision Point:** The system checks contextual information such as previous health records or environmental conditions before scheduling reminders.
- **System Actions:** Users receive reminders, complete tasks, or postpone them. Overdue tasks trigger follow-up reminders.

Figure 3.4: Care Task Activity Diagram

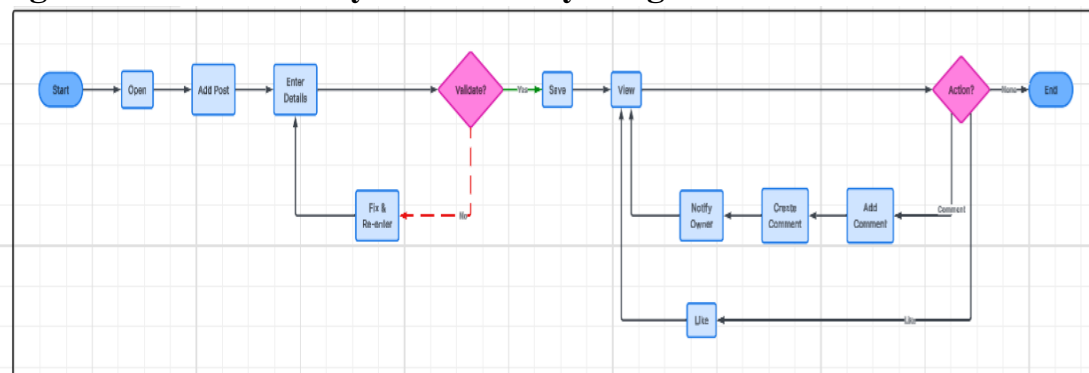


3.4.3 Community Post Activity Diagram

This diagram illustrates how users interact with the community module.

- **Workflow:** Users create posts by entering titles, descriptions, and images.
- **Decision Point:** The system validates the post before saving.
- **System Actions:** Other users can view, like, and comment on posts. Notifications are generated when comments are added.

Figure 3.5: Community Post Activity Diagram



3.5 Sequence Diagram

The Sequence Diagram represents the interaction between different components of the Plant Care Scheduler application over time. It illustrates how the Controller, Service, Repository, Database, and Notification components communicate during important business processes.

3.5.1 Care Task Sequence Diagram

The sequence begins when the user creates or auto-generates a care task.

The Care Task Module links the task with the selected plant and stores task information. It optionally retrieves recent health records and environmental data to schedule reminders more intelligently. After the scheduled time, reminder notifications are sent. The user either completes the task or postpones it. If the task remains incomplete, the system marks it as pending or overdue and generates follow-up reminders.

Flow:

User → Care Task Module → Plant → Health Record → Environment Data → Notification Service

Figure 3.6: Care Task Sequence Diagram



3.5.2 Community Post Sequence Diagram

The sequence begins when a user creates a community post.

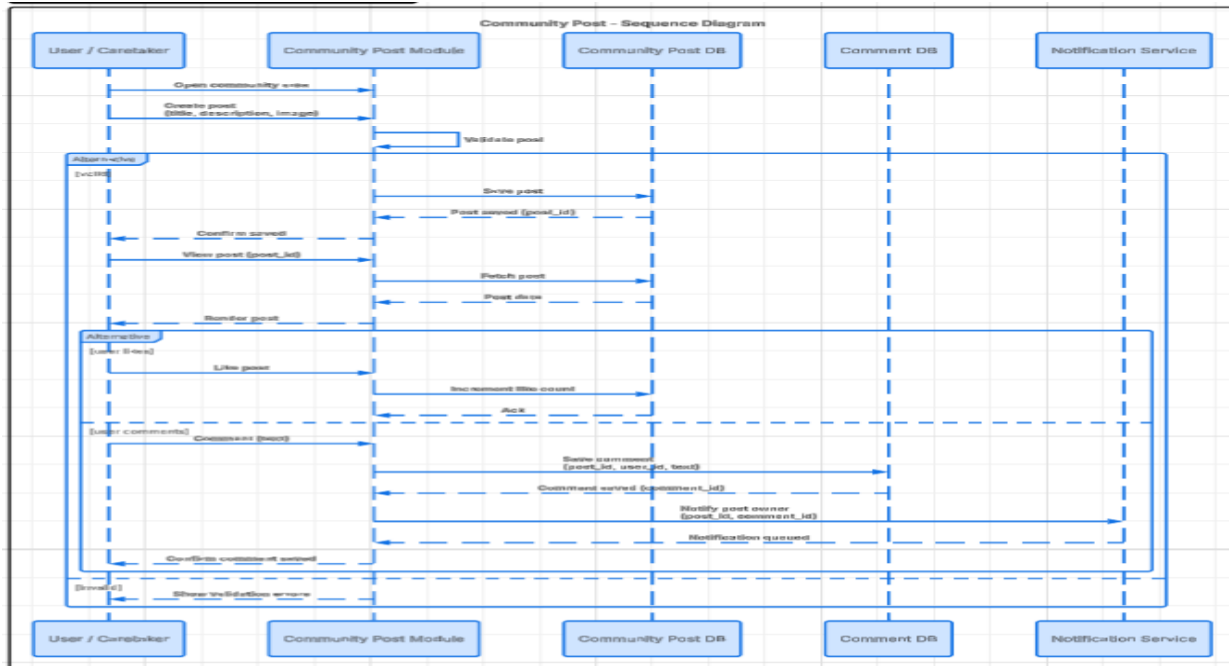
The Community Post Module validates the post and stores it in the database. Other users can view, like, and comment on the post. Comments are stored in the Comment

database, and the Notification Service informs the original post owner about new interactions.

Flow:

User → Community Post Module → Community Post Database → Comment Database → Notification Service

Figure 3.7: Community Post Sequence Diagram



Importance of the Diagrams

The UML Class Diagram illustrates the application's static structure, while the ER Diagram defines the database relationships used to store plant care information. The Activity Diagrams describe the workflows involved in plant management, care scheduling, and community interactions. The Sequence Diagrams explain how different application layers communicate during task scheduling and community activities. Together, these diagrams provide a comprehensive understanding of the Plant Care Scheduler system's architecture, business processes, and database design, improving maintainability, scalability, and future development.

CHAPTER 4: IMPLEMENTATION, TESTING AND RESULTS

4.1 Technology Stack

The **Plant Care Scheduler** is implemented as a backend REST API using **Java 17** and **Spring Boot 3.5.15**. The application follows a layered architecture consisting of Controller, Service, Repository, and Database layers to ensure modularity, maintainability, scalability, and efficient request processing.

The major technologies used in the project are:

Technology	Purpose
Java 17	Core programming language
Spring Boot 3.5.15	Backend application development
Spring Web	RESTful API development
Spring Data JPA	Database access and persistence
Hibernate	Object Relational Mapping (ORM)
Spring Security	Authentication and authorization
JWT (JSON Web Token)	Token-based authentication
BCrypt	Secure password hashing
MySQL	Relational database management system
Lombok	Reduction of boilerplate Java code
Swagger / OpenAPI	API documentation and testing
Maven	Dependency and build management
JUnit, Mockito, MockMvc	Unit, integration, and controller testing

4.2 System Implementation

The application follows a standard layered architecture to ensure separation of concerns and maintainability:

Controller Layer → Service Layer → Repository Layer → MySQL Database

- **Controller Layer:** Receives HTTP requests, validates request data, invokes business services, and returns standardized JSON responses.

- **Service Layer:** Contains the application's business logic, validates data, processes plant care operations, and coordinates communication between controllers and repositories.
- **Repository Layer:** Extends JpaRepository to perform CRUD operations on the MySQL database without requiring manual SQL queries.

The application consists of the following major modules:

- **User Management:** Handles user registration, authentication, profile management, and role assignment (USER and SPECIALIST).
- **Plant Management:** Manages plant information including species, location, growth measurements, watering history, fertilizing history, pruning history, and health status.
- **Species Management:** Maintains standard plant species information such as scientific name, watering frequency, sunlight requirements, humidity range, soil pH, fertilizer schedule, and care tips.
- **Care Task Management:** Creates, schedules, updates, and tracks plant care tasks such as watering, fertilizing, pruning, and repotting. Reminder information is also maintained.
- **Health Record Management:** Records plant health assessments including symptoms, diagnoses, treatments, specialist observations, recovery status, and follow-up schedules.
- **Environment Data Management:** Stores environmental parameters including temperature, humidity, light intensity, soil moisture, pH level, sensor details, and recording timestamps.
- **Community Management:** Enables users to create posts, upload plant-related information, interact through comments, and share gardening experiences.
- **Consultation Management:** Allows users to request consultations with specialists and manage appointment details and consultation status.
- **Notification Management:** Maintains reminders and notifications related to care tasks, consultations, and community activities.

The application exposes RESTful APIs using HTTP methods such as **POST**, **GET**, **PUT**, and **DELETE**. Data is exchanged between the client and backend primarily in **JSON** format.

4.3 Security Implementation

Application security is implemented using **Spring Security** together with **JWT (JSON Web Token)** authentication.

During user registration, passwords are securely encrypted using **BCrypt** before being stored in the database. During login, the submitted credentials are authenticated using Spring Security's Authentication Manager. Upon successful authentication, a JWT access token is generated and returned to the client.

For accessing protected REST APIs, the client sends the generated token in the HTTP request header:

Authorization: Bearer <JWT_TOKEN>

The **JwtAuthenticationFilter** intercepts every incoming request, extracts the JWT token from the Authorization header, validates the token, loads the authenticated user's details, and establishes the security context before allowing access to protected resources.

Role-based authorization is applied to ensure that only authenticated users can access protected endpoints, while registration, login, and Swagger documentation remain publicly accessible.

Security Process Flow:

Registration → BCrypt Password Encryption → Login → Credential Verification → JWT Generation → Bearer Token → JWT Validation → Authentication → Protected API Access

4.4 Testing and Results

The application is tested using **JUnit**, **Mockito**, and **MockMvc**. Testing verifies entity validation, service-layer business logic, repository operations, controller endpoints, security configuration, and exception handling.

Major Tested Scenarios

- **Model Validation**
 - Verification of entity constraints, getters, setters, and validation annotations.
- **Service Layer Testing**
 - Adding new plants.
 - Managing plant species.
 - Creating care tasks.
 - Recording health assessments.
 - Managing environmental data.
 - Creating community posts.

- Scheduling consultations.
- Managing notifications.
- **Endpoint Testing**
 - Registering users using **POST /api/auth/register**.
 - Logging in using **POST /api/auth/login**.
 - Creating plants using **POST /api/plants**.
 - Retrieving plant information using **GET /api/plants**.
 - Creating care tasks using **POST /api/caretasks**.
 - Recording health information using **POST /api/healthrecords**.
 - Managing consultations using the consultation endpoints.
- **Security Testing**
 - Successful JWT generation after valid login.
 - Accessing protected APIs using valid JWT tokens.
 - Rejecting requests with invalid or expired JWT tokens.
 - Preventing unauthorized access to secured endpoints.
- **Error Handling**
 - Returning **HTTP 400 Bad Request** for invalid request data.
 - Returning **HTTP 401 Unauthorized** for authentication failures.
 - Returning **HTTP 404 Not Found** for unavailable resources.
 - Returning **HTTP 409 Conflict** for duplicate usernames or email addresses.
- **Response Validation**
 - Verification of JSON response structures, HTTP status codes, and returned entity data.

The REST APIs are documented and tested using **Swagger UI**, providing an interactive interface for sending requests and validating responses. Successful execution of the APIs confirms the proper interaction between the Controller, Service, Repository, Security, and Database layers.

Expected Test Results

Test Scenario	Expected Result
Valid user registration	HTTP 201 Created, user registered successfully
Duplicate username/email	HTTP 409 Conflict

Test Scenario	Expected Result
Valid user login	HTTP 200 OK, JWT token generated
Invalid login credentials	HTTP 401 Unauthorized
Access protected API with valid JWT	HTTP 200 OK
Access protected API with invalid/expired JWT	HTTP 401 Unauthorized
Create plant	HTTP 201 Created, plant saved successfully
Retrieve plant details	HTTP 200 OK, requested data returned
Create care task	HTTP 201 Created
Record health assessment	HTTP 201 Created
Malformed JSON request	HTTP 400 Bad Request
Resource not found	HTTP 404 Not Found
Validation failure	HTTP 400 Bad Request

Figure 4.1: Folder Structure

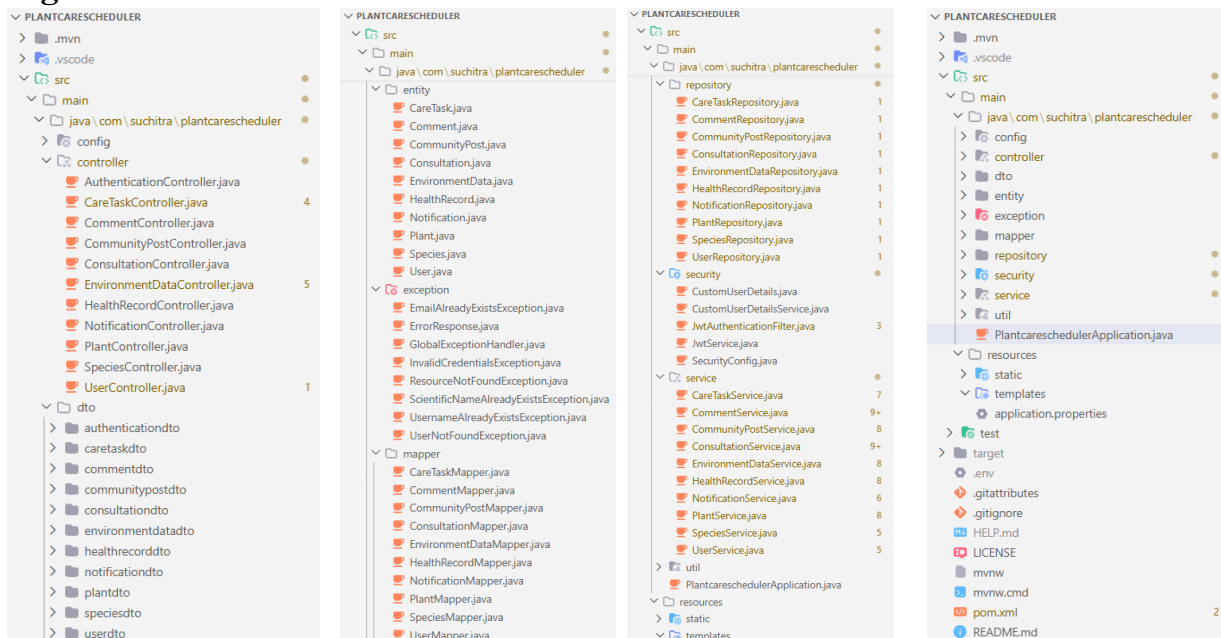
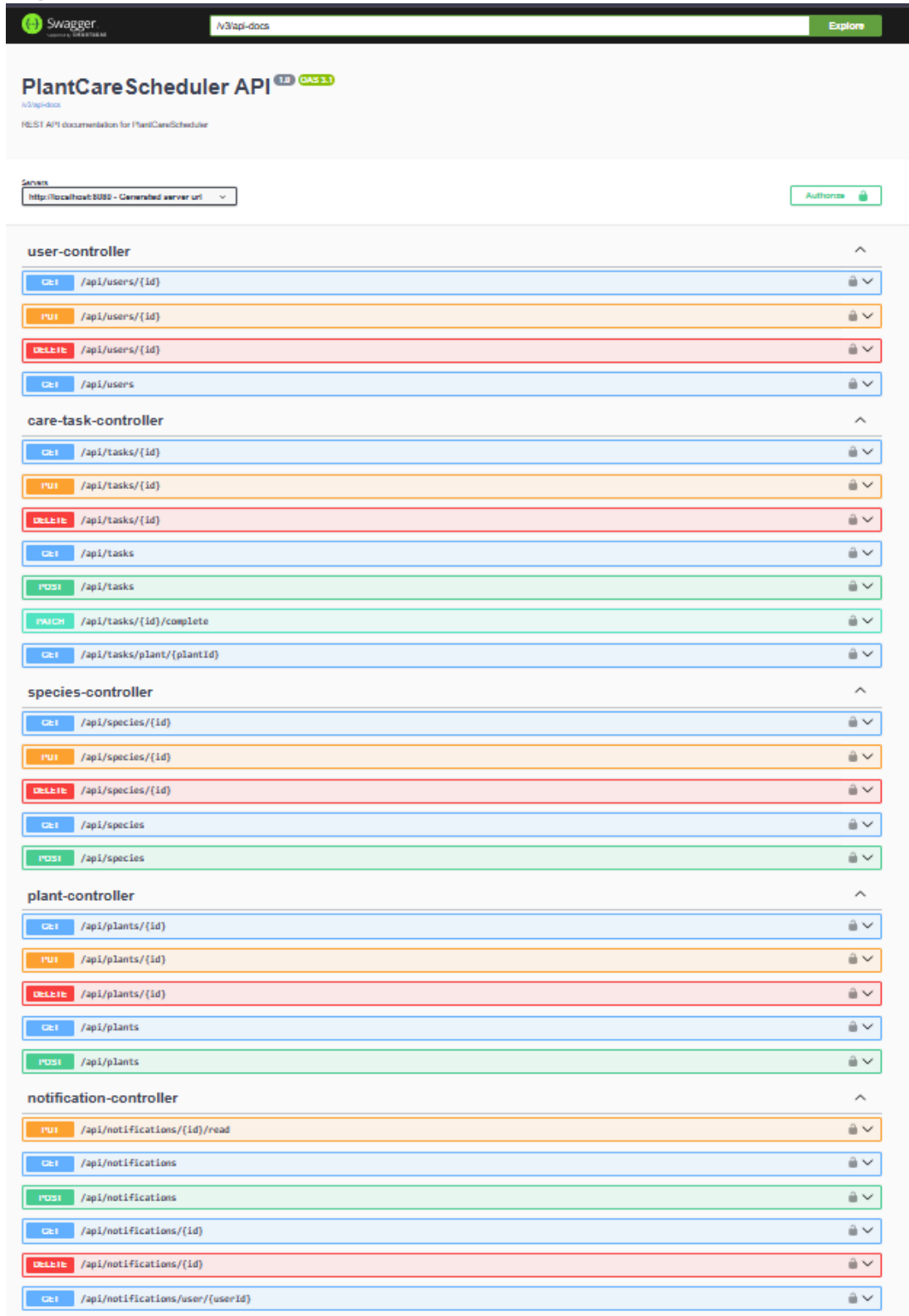


Figure 4.2: API Documentation



The image displays the Swagger UI for the PlantCareScheduler API. The interface features a top navigation bar with the Swagger logo, a search bar containing "/v3/api-docs", and an "Explore" button. Below the header, the API title "PlantCareScheduler API" is shown with version tags "1.0" and "OAS 3.1". A subtitle indicates it is "REST API documentation for PlantCareScheduler".

The main content area lists several API controllers, each with a list of endpoints and their corresponding HTTP methods. The controllers are:

- user-controller**:
 - GET /api/users/{id}
 - PUT /api/users/{id}
 - DELETE /api/users/{id}
 - GET /api/users
- care-task-controller**:
 - GET /api/tasks/{id}
 - PUT /api/tasks/{id}
 - DELETE /api/tasks/{id}
 - GET /api/tasks
 - POST /api/tasks
 - PATCH /api/tasks/{id}/complete
 - GET /api/tasks/plant/{plantId}
- species-controller**:
 - GET /api/species/{id}
 - PUT /api/species/{id}
 - DELETE /api/species/{id}
 - GET /api/species
 - POST /api/species
- plant-controller**:
 - GET /api/plants/{id}
 - PUT /api/plants/{id}
 - DELETE /api/plants/{id}
 - GET /api/plants
 - POST /api/plants
- notification-controller**:
 - PUT /api/notifications/{id}/read
 - GET /api/notifications
 - POST /api/notifications
 - GET /api/notifications/{id}
 - DELETE /api/notifications/{id}
 - GET /api/notifications/user/{userId}

Each endpoint entry includes a color-coded method label (GET, PUT, DELETE, POST, PATCH), the full URL, and a lock icon with a dropdown arrow on the right. A "Servers" section at the top left shows a default server URL: "http://localhost:3030 - Generated server url". An "Authorize" button is located in the top right corner of the main content area.

Figure 4.3: Portal testcases for Plant Care Scheduler

Test Case Results ✕

Total Count : 20 ✓ Success Count : 10 ✗ Failure Count : 10

Type	Test Case	Result	Match Percentage
Maven Junit	SpringBoot_DevelopCoreAPIsAndBusinessLogic_testGetAllPlants	Success	-
Maven Junit	SpringBoot_DevelopCoreAPIsAndBusinessLogic_testGetCarePlan_WaterMethod	Success	-
Maven Junit	SpringBoot_DevelopCoreAPIsAndBusinessLogic_testGetCarePlan_SunlightMethod	Success	-
Maven Junit	SpringBoot_DevelopCoreAPIsAndBusinessLogic_testDeletePlant_Found	Success	-
Maven Junit	SpringBoot_DevelopCoreAPIsAndBusinessLogic_testDeletePlant_NotFound	Success	-
Maven Junit	SpringBoot_DevelopCoreAPIsAndBusinessLogic_testGetAllPlants_EmptyList	Success	-
Maven Junit	SpringBoot_DevelopCoreAPIsAndBusinessLogic_testAddPlant_NullBody	Success	-
Maven Junit	SpringBoot_DevelopCoreAPIsAndBusinessLogic_testGetCarePlan_NoMethodParam	Success	-
Maven Junit	SpringBoot_DevelopCoreAPIsAndBusinessLogic_testGetCarePlan_InvalidMethodReturnInserted	Success	-

CHAPTER 5: CONCLUSION AND FUTURE ENHANCEMENTS

5.1 Conclusion

The **Plant Care Scheduler** provides a centralized and secure platform for managing various aspects of plant care and monitoring. The system integrates essential modules including user management, plant management, species management, care task scheduling, health record management, environmental monitoring, community interaction, specialist consultation, and notification management within a single application.

The application is developed using **Java 17**, **Spring Boot 3.5.15**, and **MySQL**, while ensuring robust application security through **Spring Security**, **JWT authentication**, and **BCrypt password hashing**. By adopting a layered architecture consisting of **Controller**, **Service**, and **Repository** layers, the application achieves improved modularity, maintainability, scalability, and efficient database interaction. Overall, Plant Care Scheduler provides a reliable and extensible foundation for digital plant care management, enabling users to monitor plant health, automate care activities, and collaborate with the gardening community through a secure RESTful backend.

5.2 Future Enhancements

The application can be further enhanced by incorporating the following features to improve functionality and user experience:

- **AI-Based Plant Disease Detection:** Integrate machine learning models to identify plant diseases from uploaded leaf images and provide treatment recommendations.
- **Automated Notification Services:** Integrate email, SMS, or push notification services to remind users about watering, fertilizing, pruning schedules, consultation appointments, and important plant health updates.
- **Image Gallery and Plant Recognition:** Allow users to upload plant images for automatic species identification and maintain a visual growth history of each plant.

Cloud Deployment and Backup: Deploy the application on cloud platforms with automated database backup and synchronization to improve availability, scalability, and data security.